TUGAS : STRUKTUR DATA

NAMA : RISQKA ALAMSYAH KONDAKH

NIM : 257111079

A. Linked List Codingan

```
class Node:
    def __init__(self, nim, nama, nilai):
        self.nim = nim
        self.nama = nama
        self.nilai = nilai
        self.next = None

class ManajemenMahasiswaSLL:
    def __init__(self):
        self.head = None
        self.jumlah = 0

    def insert_awal(self, nim, nama, nilai):
        new_node = Node(nim, nama, nilai)
        new_node.next = self.head
        self.head = new_node
        self.jumlah += 1
        print(f"✅ {nama} berhasil ditambah di AWAL. [O(1)]")

    def insert_akhir(self, nim, nama, nilai):
        new_node = Node(nim, nama, nilai)
        if not self.head:
            self.head = new_node
        else:
            curr = self.head
            while curr.next:
                curr = curr.next
            curr.next = new_node
        self.jumlah += 1
        print(f"✅ {nama} berhasil ditambah di AKHIR. [O(n)]")

    def insert_urut_nim(self, nim, nama, nilai):
        new_node = Node(nim, nama, nilai)
        if not self.head or nim < self.head.nim:
            new_node.next = self.head
            self.head = new_node
```

```
            self.jumlah += 1
            print(f"✅ {nama} ditambah terurut di posisi awal.")
            return
        curr = self.head
        while curr.next and curr.next.nim < nim:
            curr = curr.next
        new_node.next = curr.next
        curr.next = new_node
        self.jumlah += 1
        print(f"✅ {nama} ditambah terurut di tengah/akhir.")

    def delete_by_nim(self, nim):
        if not self.head:
            print("❌ List kosong!")
            return
        if self.head.nim == nim:
            hapus = self.head
            self.head = self.head.next
            self.jumlah -= 1
            print(f"🗑️ Data {hapus.nama} (NIM: {nim}) berhasil dihapus.")
            return
        curr = self.head
        prev = None
        while curr and curr.nim != nim:
            prev = curr
            curr = curr.next
        if not curr:
            print(f"❌ NIM {nim} tidak ditemukan!")
            return
        prev.next = curr.next
        self.jumlah -= 1
        print(f"🗑️ Data {curr.nama} (NIM: {nim}) berhasil dihapus. [O(n)]")

    def search_by_nim(self, nim):
        curr = self.head
        posisi = 0
        while curr:
            if curr.nim == nim:
                print(f"✅ DITEMUKAN di posisi {posisi}: NIM={curr.nim},
Nama={curr.nama}, Nilai={curr.nilai}")
                return curr
            curr = curr.next
            posisi += 1
        print(f"❌ NIM {nim} tidak ditemukan! [O(n)]")
        return None
```

```
    def display(self):
        if not self.head:
            print("⚠ Data kosong.")
            return
        print("\n" + "="*60)
        print(f"{'NIM':<10} {'NAMA':<20} {'NILAI':<5} {'POSISI'}")
        print("="*60)
        curr = self.head
        idx = 0
        while curr:
            print(f"{curr.nim:<10} {curr.nama:<20} {curr.nilai:<5} [{idx}]")
            curr = curr.next
            idx += 1
        print("="*60)
        print(f"Total Data: {self.jumlah} mahasiswa")
        curr = self.head
        visual = ""
        while curr:
            visual += f"[{curr.nim}] -> "
            curr = curr.next
        visual += "NULL"
        print(f"Struktur Link: {visual}\n")

    def to_array(self):
        arr = []
        curr = self.head
        while curr:
            arr.append({"nim": curr.nim, "nama": curr.nama, "nilai": curr.nilai})
            curr = curr.next
        return arr


def main():
    mhs = ManajemenMahasiswaSLL()
    mhs.insert_akhir(257111079, "Qika", 85)
    mhs.insert_akhir(257111071, "Fika", 90)
    mhs.insert_akhir(257111057, "Tara", 88)

    while True:
        print("""
╔═══════════════════════════════════════╗
║  PROJECT STRUKTUR DATA - SLL MANAGER  ║
╠═══════════════════════════════════════╣
║ 1. Tampil Semua Data                  ║
```

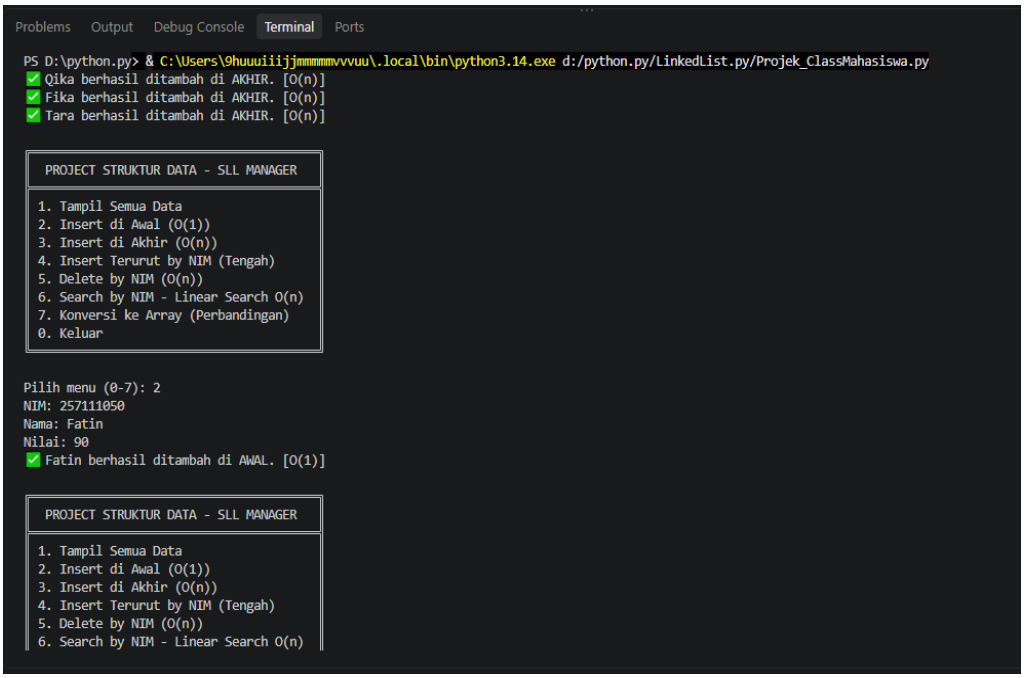
```
╠ 2. Insert di Awal (O(1))               ╣
╠ 3. Insert di Akhir (O(n))              ╣
╠ 4. Insert Terurut by NIM (Tengah)      ╣
╠ 5. Delete by NIM (O(n))                ╣
╠ 6. Search by NIM - Linear Search O(n)  ╣
╠ 7. Konversi ke Array (Perbandingan)    ╣
╠ 0. Keluar                              ╣
╚═════════════════════════════════════════╝
        """)
        pilihan = input("Pilih menu (0-7): ")

        if pilihan == "1":
            mhs.display()
        elif pilihan == "2":
            nim = int(input("NIM: "))
            nama = input("Nama: ")
            nilai = int(input("Nilai: "))
            mhs.insert_awal(nim, nama, nilai)
        elif pilihan == "3":
            nim = int(input("NIM: "))
            nama = input("Nama: ")
            nilai = int(input("Nilai: "))
            mhs.insert_akhir(nim, nama, nilai)
        elif pilihan == "4":
            nim = int(input("NIM: "))
            nama = input("Nama: ")
            nilai = int(input("Nilai: "))
            mhs.insert_urut_nim(nim, nama, nilai)
        elif pilihan == "5":
            nim = int(input("Masukkan NIM yang akan dihapus: "))
            mhs.delete_by_nim(nim)
        elif pilihan == "6":
            nim = int(input("Masukkan NIM yang dicari: "))
            mhs.search_by_nim(nim)
        elif pilihan == "7":
            arr = mhs.to_array()
            print(f"\nData Array hasil konversi: {arr}")
            print("Linked List dinamis -> Array statis untuk akses O(1)")
            arr_sorted = sorted(arr, key=lambda x: x['nim'])
            print(f"Array terurut by NIM: {[x['nim'] for x in arr_sorted]}")
        elif pilihan == "0":
            print("Terima kasih! Project Selesai.")
            break
        else:
            print("Pilihan tidak valid!")
```

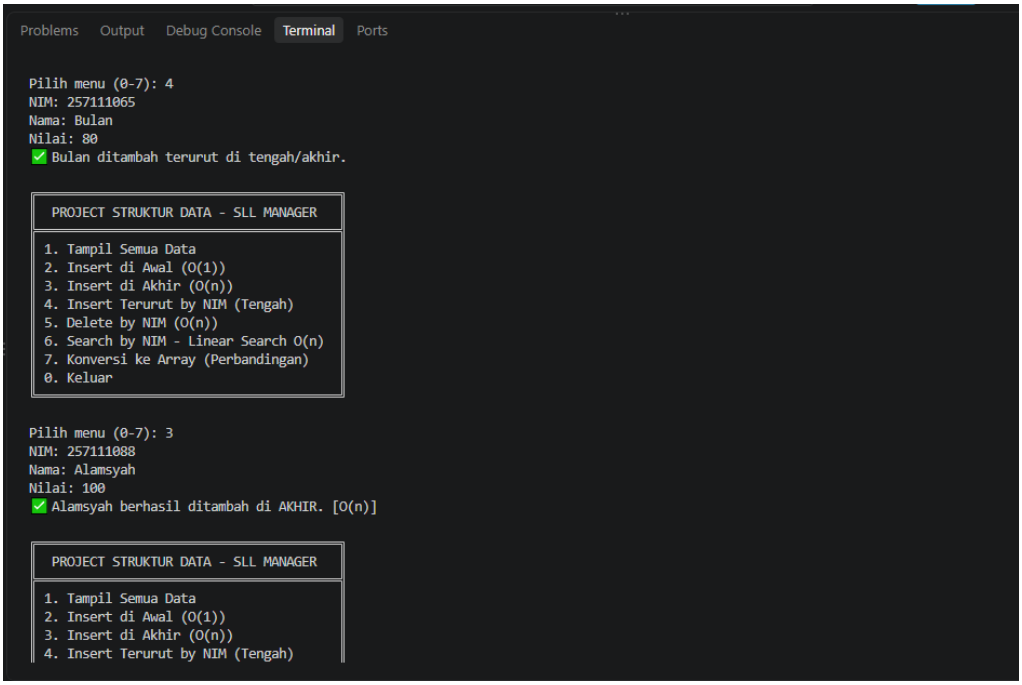
```python
if __name__ == "__main__":
    main()
```

B. HASIL

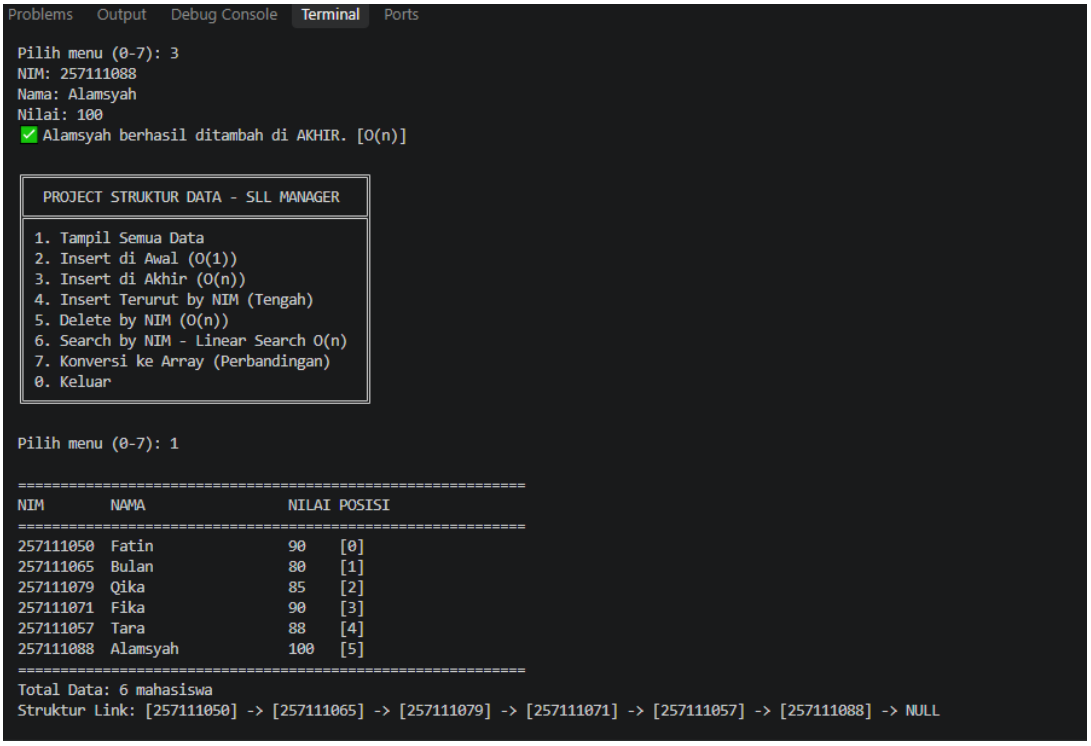
Insert awal pada Data



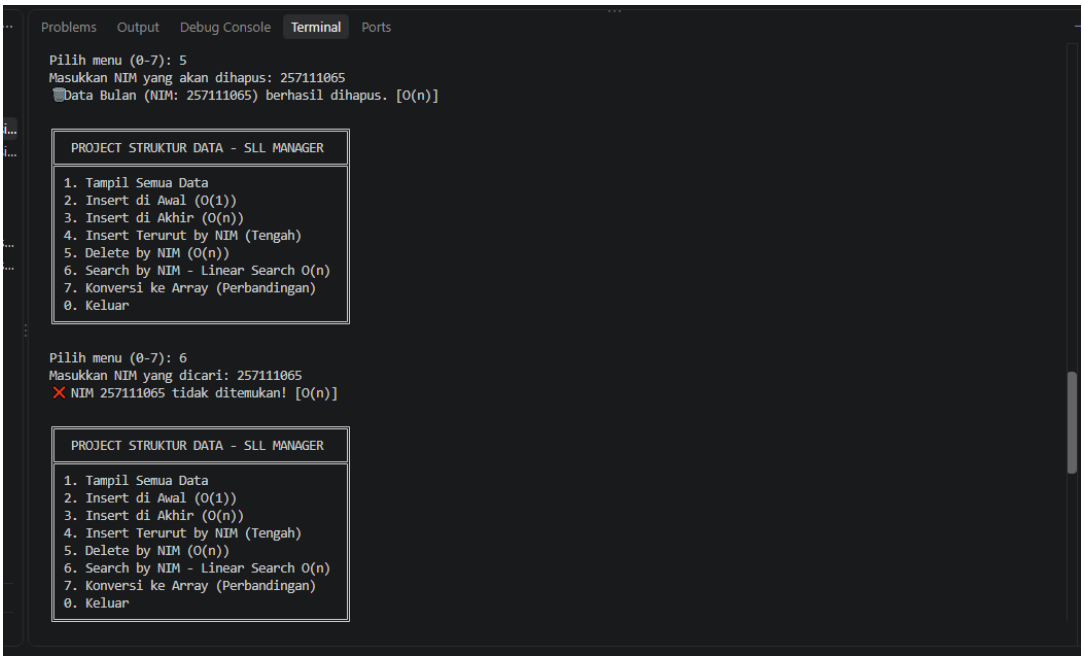
Insert Urut



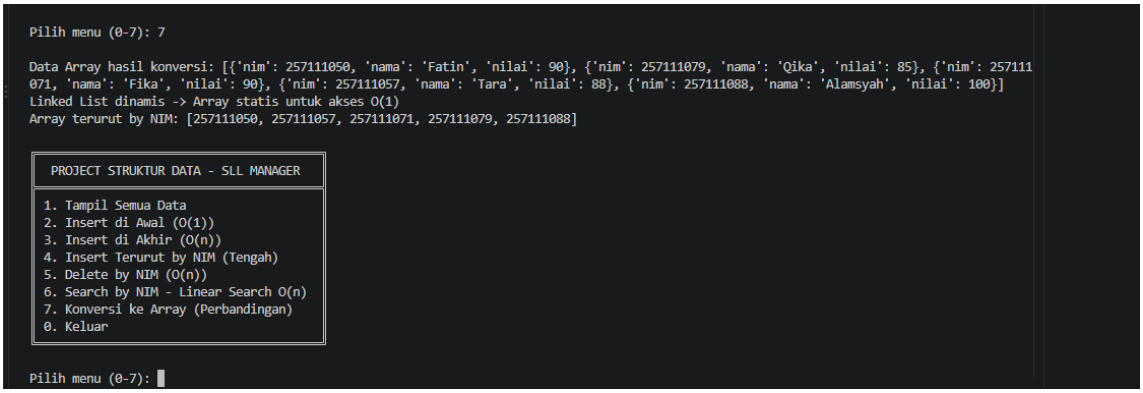
Insert Akhir dan Tampilkan Semua Data



Hapus berdasarkan NIM dan Search berdasarkan NIM



Konversi ke Array



C. Analisis Kompleksitas

1. Pilihan Implementasi
   Implementasi yang dipilih adalah Single Linked List (SLL). Alasannya: strukturnya lebih sederhana, memori per node lebih hemat karena hanya memiliki satu pointer (next), dan seluruh fitur yang diminta (tambah di awal, tengah/urut NIM, akhir, hapus, cari, dan tampil) dapat dipenuhi tanpa pointer prev. Pada fitur hapus dan cari berdasarkan NIM, penelusuran tetap harus dilakukan dari awal list, sehingga keunggulan Double Linked List tidak terlalu terasa pada kasus ini.
2. Fitur Program
   - Tambah data di awal (insert_awal)
   - Tambah data di akhir (insert_akhir)
   - Tambah data di tengah secara terurut berdasarkan NIM (insert_urut_nim)
   - Hapus data berdasarkan NIM (delete_by_nim)
   - Cari data berdasarkan NIM (search_by_nim)
   - Tampilkan semua data (display)
   - Konversi ke array sebagai pembanding (to_array)
3. Tabel Kompleksitas

Fungsi	Best Case	Worst Case	Space
Insert_awal	O(1)	O(1)	O(1)
Insert_akhir	O(1)*	O(n)	O(1)
Insert_urut_nim	O(1)	O(n)	O(1)
Delete_by_nim	O(1)	O(n)	O(1)
Search_by_nim	O(1)	O(n)	O(1)
Display	O(n)	O(n)	O(1)
To_array	O(n)	O(n)	O(n)

* Best case insert_akhir hanya terjadi saat list kosong.